Université de Bretagne Occidentale

RAPPORT DE PROJET ALGORITHMIQUE AVANCÉE

rédigé par

EL CHAMAA mohamad

JABER elham

L3 IFA 2

année 2025-2026

enseignant :

LAURENT LEMARCHAND

1. Introduction

Dans le TP précédent, le problème de couverture de réseau a été résolu de manière exacte à l'aide d'OPL et du solveur CPLEX. Cette méthode permet d'obtenir une solution optimale, mais son temps de calcul augmente fortement lorsque le nombre de points devient important.
L'objectif de ce travail est d'implanter une heuristique gloutonne afin d'obtenir une solution approchée plus rapidement, puis de comparer ses performances avec la résolution exacte.
Le travail réalisé comprend la création d'un générateur de problèmes en langage C, l'implémentation de l'heuristique en OPL, sa ré-implémentation en C, ainsi qu'une analyse expérimentale des temps de calcul et de la qualité des solutions obtenues.

2. Programme de génération des cas de test

Le programme genPoints.c a été développé afin de générer automatiquement des instances du problème de couverture au format .dat, compatible avec OPL. Il permet de produire des fichiers de test de tailles variables afin d'évaluer les performances des algorithmes.
Le programme prend en argument le nombre de points à générer, ainsi qu'une valeur optionnelle de seed permettant d'initialiser le générateur aléatoire. L'utilisation d'un seed permet d'obtenir des résultats reproductibles pour une même taille de problème.

Règles de génération des cas de test:

- Les points sont générés sous forme de grille rectangulaire.
- Les antennes sont définies avec des portées et des coûts fixes.
- La matrice de visibilité entre les points est générée aléatoirement (valeurs0ou1)
- Le nombre de points peut être modifié afin d'étudier l'impact de la taille du problème sur les performances.

Afin d'analyser le comportement des différentes approches, nous avons généré plusieurs instances de test correspondant à différentes tailles de problème : 50, 100, 150 et 200 points. Ces expérimentations permettent d'étudier l'évolution du temps de calcul ainsi que la qualité des solutions obtenues lorsque la taille du problème augmente. Elles permettent également d'évaluer la scalabilité des différentes méthodes, en observant leur capacité à traiter efficacement des instances de plus en plus grandes.

3. Heuristique gloutonne en OPL – couv2heuristique.mod

L'heuristique a été implantée dans le fichier couv2heuristique.mod.

Contrairement à la résolution exacte, cette version n'utilise pas le solveur CPLEX. L'algorithme est entièrement exécuté dans un bloc execute.

Principe de l'heuristique

L'algorithme suit une stratégie gloutonne. À chaque itération, il sélectionne l'antenne et la position qui maximisent le ratio :

$$\frac{\textit{nombre de points couverts}}{\textit{coût de l'antenne}}$$

Le fonctionnement est le suivant :

- Initialiser l'ensemble des points non couverts.
- Tant qu'il reste des points non couverts :
  - Tester toutes les antennes et toutes les positions possibles.
  - Calculer le nombre de points non couverts pouvant être couverts.
  - Calculer le ratio couverture / coût.
  - Sélectionner l'antenne ayant le meilleur ratio.
  - Retirer les points couverts.
- Calculer le coût total.

L'implémentation utilise les structures d'ensembles d'OPL (add,remove,clear, importSet, size) afin de manipuler dynamiquement les points non couverts.

```
// L Lemarchand
// EL CHAMAA mohamad && JABER elham
// Definition du tuple representant un point dans le plan
tuple Tpoint {
    int x; // Coordonne x du point
    int y; // Coordonne y du point
};
{ Tpoint } points = ...; // Ensemble de points representant les emplacements potentiels pour les antennes
float Distance[points][points];// Tableau 2D contenant les distances entre chaque paire de points


execute {// Calcul des distances entre chaque paire de points
    for (var p1 in points)
        for (var p2 in points) {
            var dx = p1.x - p2.x; // Difference en x
            var dy = p1.y - p2.y; // Difference en y
            Distance[p1][p2] = Opl.sqrt(dx*dx + dy*dy); // Calcul de la distance euclidienne
        }
}
tuple Tantenne {
    float portee; // Portee de l'antenne
    int cout;     // Cout de l'antenne
};
{ Tantenne } antennes = ...;// Ensemble d'antennes avec leurs portees et les couts
int visibilite[points][points] = ...;// Matrice binaire indiquant la visibilite entre chaque point et chaque antenne
{Tpoint} nonCouverts;    // Points non encore couverts
{Tpoint} tmpCouverts;    // Sauvegarde des points couverts par une antenne dans une iteration
{Tpoint} bestCouverts;   // Meilleur ensemble couvert courant
int select[antennes][points]; // Solution
float cout = 0.0;        // Cout total
execute {// Heuristique
    for (var a in antennes)// Initialisation de la solution a 0
        for (var p in points)
            select[a][p] = 0;
    nonCouverts.importSet(points); // Au depart tous les points doivent etre couverts
    var taille = nonCouverts.size; // Variable qui contient le nombre de points restant a couvrir
    while (taille != 0) { // On continue tant qu'il reste des points non couverts
        var max = -1.0; // Initialisation du maximum, stocke le meilleur ratio (couverture / cout)
        var max_a;      // Antenne correspondant au meilleur ratio
        var max_pa;     // Meilleur point
        for (var a in antennes) {// On teste toutes les antennes disponibles
            for (var pa in points) { // Pour chaque antenne, on teste toutes les positions possibles
                tmpCouverts.clear(); // Nettoyage de tmpCouverts a chaque iteration
                for (var p in nonCouverts) {// Parcours des points non couverts
                    if ((Distance[pa][p] <= a.portee) && (visibilite[pa][p] == 1)) {// La distance est inferieure a la portee et les deux points sont visibles l'un de l'autre
                        tmpCouverts.add(p); // Si le point est couvert, on l'ajoute
                    }
                }
                if (tmpCouverts.size > 0) {// Calcul du rapport nombre de points couverts / cout de l'antenne
                    var ratio = tmpCouverts.size * 1.0 / a.cout; // On force le calcul en float pour eviter la division entiere
                    if (ratio > max) {
                        max = ratio;       // Mise a jour du maximum
                        max_a = a;         // Mise a jour de l'antenne
                        max_pa = pa;       // Mise a jour du point
                        bestCouverts.clear(); // Nettoyage de bestCouverts
                        bestCouverts.importSet(tmpCouverts); // Mise a jour de bestCouverts
                    }
                }
            }
        }
        for (var p in bestCouverts) { // Suppression des points couverts et mise a jour de la solution
            nonCouverts.remove(p); // Suppression du point de nonCouverts
        }
        select[max_a][max_pa] = 1; // Selection de l'antenne pour le point
        cout = cout + max_a.cout;  // Mise a jour du cout total
        taille = nonCouverts.size; // Mise a jour de la taille de nonCouverts
    }
    writeln("cout = ", cout);// Affichage du cout pour compatibilite avec les scripts shell
}
execute {// Sortie du resultat detaille
    var sortie = "couv2.res"; // Fichier de sortie
    var ofile = new IloOplOutputFile(sortie); // Ouverture du fichier de sortie
    ofile.writeln("cout = ", cout); // Ecriture du cout total
    for (var a in antennes)
        for (var pa in points)
            if (select[a][pa] == 1)
                ofile.writeln(a.portee, ", ", pa.x, " ", pa.y); // Ecriture des details des antennes selectionnees
    ofile.close(); // Fermeture du fichier de sortie
    writeln("trace dans ", sortie); // Affichage du chemin du fichier de sortie
}
// Il est possible d'eviter l'appel au solver en ecrivant un main() a la place du dernier execute -- cf exemples OPL
```

cou2heuristique.mod

4. Heuristique gloutonne en C – heuristique.c

Après avoir implémenté l'heuristique en OPL, nous avons ré-implanté le même algorithme en langage C afin de comparer les performances entre une version interprétée (OPL) et une version compilée.

Le principe de l'heuristique reste identique : à chaque itération, on sélectionne l'antenne et la position qui maximisent le ratio :

$$\frac{\textit{nombre de points couverts}}{\textit{coût de l'antenne}}$$

Le programme se déroule en plusieurs étapes :

- Lecture des données à partir du fichier c.dat.
- Initialisation d'un tableau indiquant les points non couverts.
- Recherche, pour chaque antenne et chaque position possible, du nombre de points pouvant être couverts.
- Calcul du ratio couverture / coût.
- Sélection de l'antenne maximisant ce ratio.
- Mise à jour des points couverts.
- Répétition jusqu'à ce que tous les points soient couverts.

La version en C permet une exécution plus rapide grâce à la compilation du programme.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#define MAXANTENNES 10
#define MAXPOINTS 1000
typedef struct {
    float portee;
    int cout;
} antenne_t
int NbAntennes;
antenne_t Antennes[MAXANTENNES];
int NbPoints;
float Distances[MAXPOINTS][MAXPOINTS];
int Visibilite[MAXPOINTS][MAXPOINTS];
void readCouverture(char *filename)
{
    FILE *fp = fopen(filename, "r");// Ouverture du fichier de donnees
    if (!fp) exit(0);
    fscanf(fp, "%d", &NbAntennes);// Lecture du nombre d'antennes
    for (int a=0; a<NbAntennes; a++)// Lecture des portees et couts des antennes
        fscanf(fp, "%f%d", &(Antennes[a].portee),
                &(Antennes[a].cout));
    fscanf(fp, "%d", &NbPoints); // Lecture du nombre de points
    for (int i=0; i<NbPoints; i++)// Lecture des matrices Distance et Visibilite
        for (int j=0; j<NbPoints; j++)
            fscanf(fp, "%f%d",
                &(Distances[i][j]),
                &(Visibilite[i][j]));
    fclose(fp);
}
// Heuristique gloutonne : A chaque iteration on choisit l'antenne et la position
// qui maximisent le ratio (nombre de points couverts / cout)
float couvrir(int select[MAXANTENNES][MAXPOINTS])
{
    float cout = 0.0;
    int nonCouverts[MAXPOINTS];
    for (int i = 0; i < NbPoints; i++)  // Tous les points sont non couverts au début
        nonCouverts[i] = 1;
    while (1)
    {
        int reste = 0;
        for (int i = 0; i < NbPoints; i++)// Vérifier s'il reste des points non couverts
            if (nonCouverts[i]) reste = 1;
        if (!reste) break;
        int bestAntenne = -1;
```

```c
        if (nonCouverts[i]) reste = 1;
    if (!reste) break;
    int bestAntenne = -1;
    int bestPoint = -1;
    float bestRatio = -1.0;
    for (int a = 0; a < NbAntennes; a++)// Parcourir toutes les antennes disponibles
    {
        for (int pa = 0; pa < NbPoints; pa++)// Tester chaque point comme emplacement possible pour une antenne
        {
            int couverts = 0;
            for (int p = 0; p < NbPoints; p++)// Vérifier quels points non couverts peuvent être couverts par cette antenne
            {
                if (nonCouverts[p] &&
                    Distances[pa][p] <= Antennes[a].portee &&
                    Visibilite[pa][p])
                {
                    couverts++;
                }
            }
            if (couverts > 0)// Calculer le rapport couverture/coût
            {
                float ratio = (float)couverts / Antennes[a].cout;
                if (ratio > bestRatio)// Garder la meilleure antenne trouvée
                {
                    bestRatio = ratio;
                    bestAntenne = a;
                    bestPoint = pa;
                }
            }
        }
    }
    if (bestAntenne == -1) break;// Si aucune antenne na ete trouvee, arreter
    select[bestAntenne][bestPoint] = 1;// Sélectionner cette antenne
    cout += Antennes[bestAntenne].cout;
    for (int p = 0; p < NbPoints; p++)// Mettre à jour les points couverts
    {
        if (nonCouverts[p] &&
            Distances[bestPoint][p] <= Antennes[bestAntenne].portee &&
            Visibilite[bestPoint][p])
        {
            nonCouverts[p] = 0;
        }
    }
}
return cout;
```

heuristique.c

```c
int main(int argc, char *argv[])
{
    char *filename = "c.dat";// Nom du fichier par défaut
    clock_t start, end;
    double cpu_time_used;
    start = clock();// Enregistrez le temps de début
    if (argc == 2)// Vérifier si un nom de fichier est passé en argument
    {
        filename = argv[1];
        char cmd[1024];
        sprintf(cmd,
            "LD_LIBRARY_PATH=/home1/commun_deptinfo/enseignants/lemarchand/CPLEX_Studio126/opl/bin/x86-64_linux:/home1/commun_deptinfo/enseignants/lemarchand/CPLEX_Studio126/opl/lib/x86-64_linux/static_p
            "%s/oplrun dat2prog.mod %s",
            "/home1/commun_deptinfo/enseignants/lemarchand/CPLEX_Studio126/opl/bin/x86-64_linux",
            argv[1]);
        system(cmd);
        sleep(1);

    }
    // Appeler la fonction pour lire les données du fichier (en mode local on lit directement le fichier passé en argument)
    // Appeler la fonction pour lire les données du fichier "c.dat"
    readCouverture("c.dat");
    printf("from %s: read %d points and %d antennas problem\n",
            filename, NbPoints, NbAntennes);
    int select[MAXANTENNES][MAXPOINTS];// Tableau pour stocker les antennes sélectionnées pour chaque point
    for (int a = 0; a < MAXANTENNES; a++)// Initialisation à 0
        for (int p = 0; p < MAXPOINTS; p++)
            select[a][p] = 0;
    float cout = couvrir(select);// Appeler la fonction pour couvrir les points avec les antennes
    for (int a = 0; a < NbAntennes; a++)// Afficher les antennes sélectionnées pour chaque point
    {
        for (int p = 0; p < NbPoints; p++)
        {
            if (select[a][p] == 1)
            {
                printf("\nant %d at point %d\n", a + 1, p + 1);
            }
        }
        printf("\n");
    }
    printf("cost : %.2f\n", cout);// Afficher le coût total de la solution
    end = clock();// Enregistrez le temps de fin
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Le temps d'exécution est %f secondes.\n", cpu_time_used);
    return 0;
```

Cette implémentation permet d'obtenir les mêmes résultats que la version OPL, mais avec un temps d'exécution généralement plus faible.

5. Script shell (Automatisation des tests)

Afin de comparer de manière systématique les différentes méthodes de résolution, un script shell a été développé.

Ce script permet d'automatiser les étapes suivantes :

- génération d'instances de tailles différentes (50, 100, 150 et 200 points),
- exécution de la résolution exacte avec OPL,
- exécution de l'heuristique en OPL,
- exécution de l'heuristique en langage C,
- mesure des temps d'exécution,
- enregistrement des résultats dans un fichier resultat_final.csv.

L'automatisation permet d'obtenir des résultats reproductibles et facilite la comparaison des performances.

```bash
#!/bin/bash
# Configuration CPLEX
export cplexdir=/home1/commun_depinfo/enseignants/lemarchand/CPLEX_Studio126
export LD_LIBRARY_PATH=$cplexdir/opl/bin/x86-64_linux:$cplexdir/opl/lib/x86-64_linux/shared_pic:$cplexdir/cplex/lib/x86-64_linux/static_pic
export oplrun=$cplexdir/opl/bin/x86-64_linux/oplrun
# création du fichier résultat
rm -f resultat_final.csv
echo "Points,CoutExact,CoutHeurOPL,CoutHeurC,TempsExact,TempsHeurOPL,TempsHeurC" > resultat_final.csv
for n in 50 100 150 200
do
    echo "----- Taille $n ------"
    ./genPoints $n 1 > tmp.dat
    #  EXACT
    /usr/bin/time -f "%e" -o timeExact.txt \
    $oplrun couvExact.mod tmp.dat > exact.txt 2>/dev/null
    coutExact=$(grep "cout =" exact.txt | awk '{print $3}')
    tempsExact=$(cat timeExact.txt)
    # HEUR OPL
    /usr/bin/time -f "%e" -o timeHeurOPL.txt \
    $oplrun couv2heuristique.mod tmp.dat > heurOPL.txt 2>/dev/null
    coutHeurOPL=$(grep "cout =" heurOPL.txt | awk '{print $3}')
    tempsHeurOPL=$(cat timeHeurOPL.txt)
    # HEUR C
    $oplrun dat2cprog.mod tmp.dat > /dev/null
    /usr/bin/time -f "%e" -o timeHeurC.txt \
    ./heuristique > heurC.txt 2>/dev/null
    coutHeurC=$(grep "cost" heurC.txt | awk '{print $3}')
    tempsHeurC=$(cat timeHeurC.txt)
    # écriture ligne csv
    echo "$n,$coutExact,$coutHeurOPL,$coutHeurC,$tempsExact,$tempsHeurOPL,$tempsHeurC" >> resultat_final.csv
done
rm -f tmp.dat exact.txt heurOPL.txt heurC.txt \
```

```bash
    tempsHeurC=$(cat timeHeurC.txt)
    # écriture ligne csv
    echo "$n,$coutExact,$coutHeurOPL,$coutHeurC,$tempsExact,$tempsHeurOPL,$tempsHeurC" >> resultat_final.csv
done
rm -f tmp.dat exact.txt heurOPL.txt heurC.txt \
timeExact.txt timeHeurOPL.txt timeHeurC.txt c.dat

echo "Résultats enregistrés dans resultat_final.csv"
```

doTests_final.sh

```
./genPoints n seed
```

La commande ./genPoints n seed génère une instance contenant n points. Le paramètre seed permet de contrôler l'aléatoire afin de reproduire les mêmes données lors des tests.

6.Résultats expérimentaux

Suite à l'exécution du shell des résultats apparaissent dans le fichier resultat.csv , les résultats obtenus et qui sont utilisés pour analyser le travail , se trouvent ci-joint sous forme d'une capture d'écran

resultat_final.csv

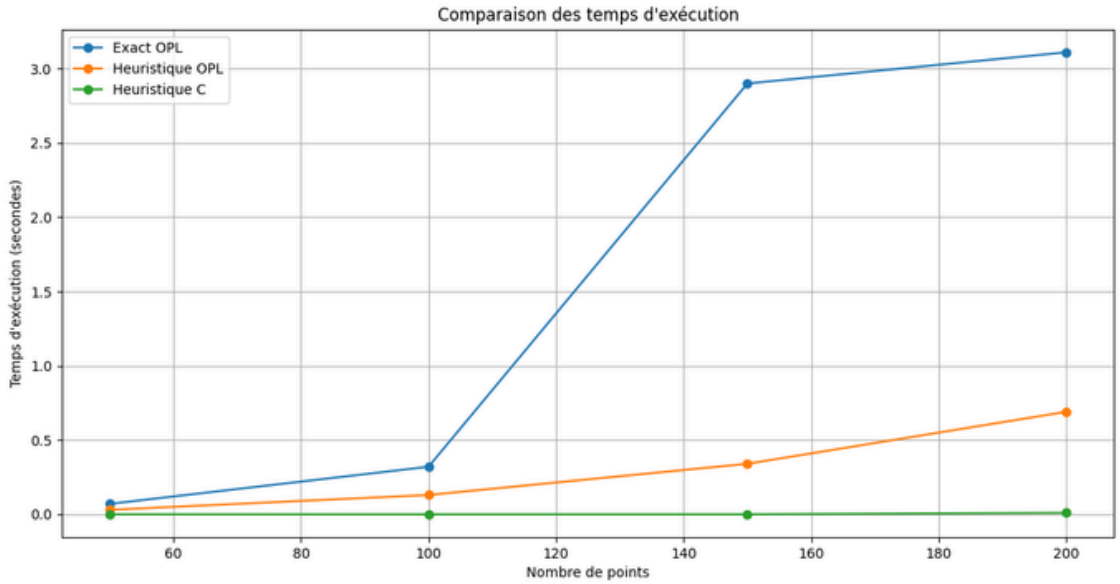
```
Points,CoutExact,CoutHeurOPL,CoutHeurC,TempsExact,TempsHeurOPL,TempsHeurC
50,700,780,780.00,0.07,0.03,0.00
100,1360,1560,1560.00,0.32,0.13,0.00
150,1900,2200,2200.00,2.90,0.34,0.00
200,2480,2880,2880.00,3.11,0.69,0.01
```

6.1 Évolution des temps de résolution

La figure ci-dessous présente l'évolution des temps d'exécution en fonction du nombre de points pour les trois méthodes étudiées :

- la résolution exacte avec OPL,
- l'heuristique gloutonne implémentée en OPL,
- l'heuristique gloutonne implémentée en C.

	A	B	C	D
1	Points	TempsExact	TempsHeurOPL	TempsHeurC
2	50	0.07	0.03	0.00
3	100	0.32	0.13	0.00
4	150	2.90	0.34	0.00
5	200	3.11	0.69	0.01



On observe que le temps de résolution exacte augmente fortement lorsque le nombre de points augmente. Cette croissance est due à la nature combinatoire du problème de couverture, qui devient plus complexe à mesure que la taille de l'instance augmente.

Les deux heuristiques présentent des temps de calcul nettement plus faibles.

L'heuristique en OPL reste rapide même pour 200 points, tandis que l'implémentation en C est la plus performante et reste quasiment instantanée pour toutes les tailles testées.

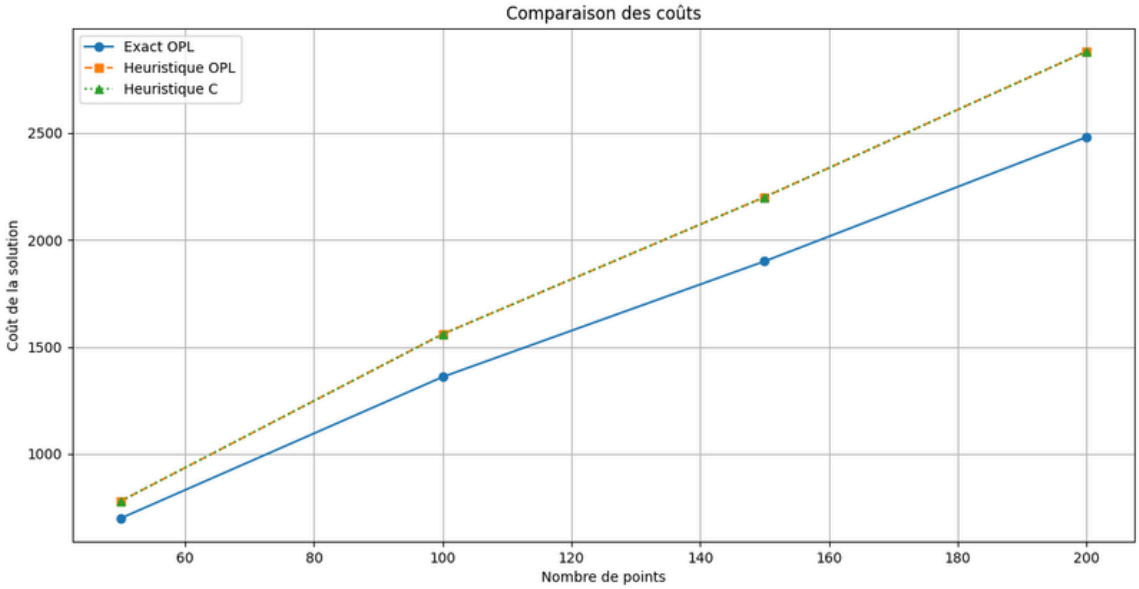
La différence de performance entre l'heuristique OPL et l'heuristique C s'explique par le fait que la version C est compilée et ne dépend pas de l'environnement OPL.

Ces résultats montrent clairement l'intérêt des approches heuristiques pour traiter des instances de grande taille.

6.Résultats expérimentaux

6.2 Qualité des solutions obtenues

La figure suivante présente la comparaison des coûts obtenus par la résolution exacte et par les deux heuristiques.



	A	B	C	D
1	Points	CoutExact	CoutHeurOPL	CoutHeurC
2	50	700	780	780
3	100	1360	1560	1560
4	150	1900	2200	2200
5	200	2480	2880	2880

On observe que les heuristiques produisent systématiquement un coût supérieur à celui de la solution optimale obtenue par la méthode exacte.

L'écart entre la solution exacte et la solution heuristique augmente légèrement lorsque le nombre de points augmente. Cela s'explique par le caractère glouton de l'algorithme, qui effectue un choix local optimal à chaque étape sans garantir l'optimalité globale.

On remarque également que les heuristiques OPL et C donnent exactement les mêmes coûts, ce qui confirme que l'implémentation en C reproduit correctement l'algorithme développé en OPL.

Ces résultats montrent que l'heuristique fournit une solution approchée de bonne qualité tout en réduisant fortement le temps de calcul.

6.Analyse globale des résultats

6.1 Analyse des temps de calcul

On observe que les heuristiques produisent systématiquement un coût supérieur à celui de la solution optimale obtenue par la méthode exacte.
L'écart entre la solution exacte et la solution heuristique augmente légèrement lorsque le nombre de points augmente. Cela s'explique par le caractère glouton de l'algorithme, qui effectue un choix local optimal à chaque étape sans garantir l'optimalité globale.
On remarque également que les heuristiques OPL et C donnent exactement les mêmes coûts, ce qui confirme que l'implémentation en C reproduit correctement l'algorithme développé en OPL.
Ces résultats montrent que l'heuristique fournit une solution approchée de bonne qualité tout en réduisant fortement le temps de calcul.

6.2 Analyse de la qualité des solutions

Concernant la qualité des solutions, les heuristiques produisent des coûts supérieurs à la solution optimale. Cet écart reste cependant raisonnable pour toutes les tailles testées.
L'augmentation progressive de cet écart avec la taille du problème est cohérente avec la nature gloutonne de l'algorithme. En effet, celui-ci effectue un choix local optimal à chaque itération, sans garantir l'optimalité globale.
On remarque également que les implémentations OPL et C donnent exactement les mêmes coûts, ce qui confirme la cohérence et la validité de l'algorithme ré-implémenté en C.

6.3 Interprétation générale

Ces résultats illustrent clairement le compromis classique en optimisation :

- La méthode exacte privilégie la qualité au détriment du temps de calcul.
- L'heuristique privilégie la rapidité tout en conservant une qualité acceptable.

Pour des instances de grande taille, l'approche heuristique apparaît comme une solution pertinente.

6.CONCLUSION

Ce TP avait pour objectif d'étudier le problème de couverture réseau en comparant une méthode exacte et des approches heuristiques.

La résolution exacte à l'aide d'OPL et du solveur CPLEX permet d'obtenir une solution optimale, mais son temps de calcul augmente fortement lorsque la taille du problème devient importante. Cette méthode reste donc adaptée uniquement aux instances de petite ou moyenne taille.
L'heuristique gloutonne développée dans ce TP permet d'obtenir des solutions approchées en un temps très réduit. Les résultats expérimentaux montrent que la qualité des solutions reste satisfaisante malgré un léger écart par rapport à l'optimal. De plus, l'implémentation en langage C offre les meilleures performances en termes de temps d'exécution.

Ce travail met ainsi en évidence l'intérêt des heuristiques pour résoudre efficacement des problèmes d'optimisation de grande taille, en acceptant un compromis raisonnable entre qualité et rapidité.

Nous vous remercions pour votre attention.